IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2020

EIE PART II: MEng, BEng and ACGI

# SOFTWARE ENGINEERING 2: OBJECT-ORIENTED SOFTWARE ENGINEERING

Monday, 1 June 10:00 am

Time allowed: 2:00 hours

**There are THREE questions on this paper.**

**Answer ALL questions.**
**Q1 and Q3 carry equal marks (30% each). Questions 2 carries 40% of the marks.**

**Any special instructions for invigilators and information for candidates are on page 1.**

Examiners responsible First Marker(s) : J.V. Pitt
Second Marker(s) : S.A. Evangelou

© Imperial College London

# ELEC95021 (EE2-12) OBJECT-ORIENTED SOFTWARE ENGINEERING

1. a) In the context of C++, and using code to illustrate your answers as appropriate:

   (*i*) What is a destructor?
   (*ii*) When is a destructor called?
   (*iii*) How are destructors different from normal member functions?
   (*iv*) How many destructors can be defined for any one class?
   (*v*) Why is it needed to write user-defined destructors?
   (*vi*) Why are objects always destroyed in reverse order of their creation?

   [ 10 ]

   b) In the context of C++, explain the following terms, using UML (Unified Modeling Language) and/or example code to illustrate your answer:

   (*i*) Abstraction;
   (*ii*) Encapsulation;
   (*iii*) Inheritance.

   [ 10 ]

   c) In the context of C++, explain the following concepts, using code to illustrate your answer:

   (*i*) Polymorphism (definition only, no code needed);
   (*ii*) Compile-time polymorphism using function overloading;
   (*iii*) Compile-time polymorphism using operator overloading;
   (*iv*) Run-time polymorphism using function over-riding.

   [ 10 ]

[ Q1 Total: 30 ]

2. In C++, `vector` is a sequence container representing arrays that can change in size. Useful public member functions are:

```
push_back(...)  //add element to the end of the vector
erase(...)      //erase element at an index
at(...)         //access element at an index
```

In C++, `pair` is a class which couples together a pair of values, which may be of different types. The individual values can be constructed and accessed through its public member functions:

```
make_pair(x,y)  //construct a pair object with its first element
                //set to x and its second element set to y
first           //access value of first element of the pair
second          //access value of second element of the pair
```

a) A standard deck of playing cards consists of 4 suits (hearts, clubs, diamonds and spades) and 13 face cards, from numbers 1 (ace) to 10, and jack, queen, and king.

Declare two constants to represent this information about a deck of cards.

[ 2 ]

b) Explain (using code) how the following declaration could be used to represent a deck of cards, commenting on the suitability of this representation:

```
vector <pair<int,int>> Cards ;
```

[ 4 ]

c) Based on the declaration of Part (b), or otherwise, create a class `Deck` that can be used to represent and manipulate a deck of cards. The class should declare the necessary data members, and at least the following member functions:

```
(constructor)   //create a new deck of cards in order
dealCard(...)   //returns a single card from top of
                //deck and erases the card from the deck
cardsLeft(...)  //returns number of cards left in deck
printCard(...)  //displays a card in human readable form,
                //e.g. (4,1) is ‘‘Ace of Spades’’
```

[ 12 ]

d) Create a class `Player` that could be used to represent a player and her hand. Write two member functions:

```
dealtCard(...)  //player is dealt a card
playCard(...)   //player plays a random card from her hand
```

Write some lines of code (a code snippet) to show how a player is dealt and plays a card.

[ 6 ]

e) Write a member function `shuffle` for class `Deck` which would shuffle a deck of cards.

[ 6 ]

*[continued. . . ]*

*[continued. . . ]*

f) Modify the member function `playCard` for class `Player` that returns a card of a given suit, or a random card if the player has no cards of that suit.

[ 6 ]

g) Internally, a vector uses a dynamically-allocated array to store its elements. Vector containers may allocate some extra storage to accommodate possible growth. Comment on the advantages to the programmer of the vector container, and discuss the trade-offs (between memory-use and processing time) involved in allowing for extra storage.

[ 4 ]

[ Q2 Total: 40 ]

3. You have been hired as a developer to join a team creating some application *ShapeWorld* in C++. It is essential that you implement your code for *ShapeWorld* according to the specification and instructions that you are given.

a) Define a structure for representing an $(x, y)$-coordinate in a 2-dimensional plane.

[ 2 ]

b) Define an array for storing 4 (four) such coordinates.

[ 2 ]

c) Suppose that a rectangular shape for *ShapeWorld* requires that the shape's lower-left corner should be under the upper-left corner, the lower-right under the upper-right, the upper-right to the right of the upper-left, and the lower-right to the right of the lower-left corner.

Define a class `Shape` that can be used to store the four corners of such rectangles, assuming that the lower-left corner is stored in array element 0, upper-left in 1, upper-right in 2, and lower-right in 3. Your class should define two member functions `setCorner(...)` and `getCorner(...)` to access the coordinate of the corner corresponding to the given index.

[ 4 ]

d) Define a constructor function for class `Shape` using only two $(x, y)$-coordinates as arguments. Give a code snippet using the constructor for creating two `Shape` objects. Explain why defining a constructor function is preferable to using `setCorner(...)`.

[ 6 ]

e) Although the constructor function creates a valid `Shape` object for this application, a programmer could still misuse `setCorner(...)` to produce an invalid `Shape` object. Write a member function `validShape` that checks whether or not a `Shape` object is (still) valid.

[ 4 ]

f) Write a member function `overlappingShape(...)` that checks if a valid `Shape` object overlaps with another valid `Shape` object (returns true if the two shapes overlap). Unfortunately your client *absolutely insists* that this function is written by checking if any one of the corners of one shape is within the boundaries of the other. Give a code snippet to show how your member function would be called.

[ 6 ]

g) Write a member function `socialDistance(...)` that returns true if the corners of one valid `Shape` object are at least 2 units from the corners of another. (It can be assumed that there is a constant `MaxDist` which defines the maximum possible distance between any two shapes.)

[ 6 ]

[ Q3 Total: 30 ]